



Excelerate Virtual Internship

Data Visualization Associate – Week 1 Report

SQL File of Six Datasets

Submitted By: Team 40

Team Members:

Names	Email
Roopesh Tangudu	tanguduroopesh@gmail.com
Aneeba Tariq	aneebatariq011@gmail.com
Sumaiya Tasnim	sumaiyaa.tasnim.18@gmail.com
Huzaifa Tariq	Huzaifatariq757@gmail.com
Tania Ntimba	taniantimba@gmail.com
Radhika Talreja	Talrejaradhika19@gmail.com
Sneha Tandon	6688sneha8865@gmail.com
Gandharva Tatkare	tatkare.gandharva@gmail.com
Arekah Taj	arekah99@gmail.com

Submission Date: 11/08/2025

THIS FILE INCLUDES QUERIES WROTE TO PERFORM DATA ANALYSIS ON DATASETS PROVIDED BY EXCELERATE PLATFORM FOR DATA VISUALIZATION ASSOCIATE INTERNSHIP.

THE STRUCTURE OF THIS FILE IS:

- 1.QUERIES FOR User Dataset
- 2.QUERIES FOR Cognito Dataset
- 3.QUERIES FOR Marketing Dataset
- 4.QUERIES FOR Learner opportunity Dataset
- 5.QUERIES FOR cohort dataset
- 6.QUERIES FOR opportunity dataset

1. USER DATASET

```
select * from learners;
```

```
SELECT COUNT(*) FROM learners;  
SELECT column_name, data_type FROM information_schema.columns WHERE  
table_name = 'learners';
```

```
SELECT country, degree, institution, major  
FROM learners  
LIMIT 20;
```

```
SELECT column_name  
FROM information_schema.columns  
WHERE table_name = 'learners';
```

```
SELECT DISTINCT degree, institution  
FROM learners  
ORDER BY degree;
```

```
SELECT  
    COUNT(*) FILTER (  
        WHERE degree IS NULL OR TRIM(LOWER(degree)) IN ('null')  
    ) AS missing_degree,  
  
    COUNT(*) FILTER (  
        WHERE institution IS NULL OR TRIM(LOWER(institution)) IN ('null')  
    ) AS missing_institution,  
  
    COUNT(*) FILTER (  
        WHERE major IS NULL OR TRIM(LOWER(major)) IN ('null')  
    ) AS missing_major  
  
FROM learners;
```

```

SELECT
    COUNT(*) FILTER (
        WHERE country IS NULL OR TRIM(LOWER(country)) IN ('null', '')
    ) AS missing_country,

    COUNT(*) FILTER (
        WHERE degree IS NULL OR TRIM(LOWER(degree)) IN ('null', '')
    ) AS missing_degree,

    COUNT(*) FILTER (
        WHERE institution IS NULL OR TRIM(LOWER(institution)) IN ('null',
'',
    ) AS missing_institution,

    COUNT(*) FILTER (
        WHERE major IS NULL OR TRIM(LOWER(major)) IN ('null', '')
    ) AS missing_major

```

```

FROM learners;

```

```

SELECT column_name, data_type
FROM information_schema.columns
WHERE table_schema = 'public' AND table_name = 'learners';

```

```

SELECT * FROM public.learners LIMIT 10;
SELECT COUNT(*) FROM public.learners;

```

```

SELECT *
FROM learners
LIMIT 5;

```

```

SELECT column_name, data_type
FROM information_schema.columns
WHERE table_name = 'learners';

```

```

SELECT COUNT(*) AS total_rows
FROM learners;

```

```

SELECT country, COUNT(*) AS total
FROM learners
GROUP BY country
ORDER BY total DESC;

```

```

SELECT DISTINCT degree
FROM learners;

```

```

SELECT
    COUNT(*) AS total_learners,
    COUNT(DISTINCT country) AS distinct_countries,
    COUNT(DISTINCT degree) AS distinct_degrees,
    COUNT(DISTINCT institution) AS distinct_institutions,

```

```
    COUNT(DISTINCT major) AS distinct_majors
FROM learners;
```

```
-- Frequency distribution for countries
SELECT country, COUNT(*) AS learner_count
FROM learners
GROUP BY country
ORDER BY learner_count DESC;
```

```
-- Frequency distribution for degrees
SELECT degree, COUNT(*) AS degree_count
FROM learners
GROUP BY degree
ORDER BY degree_count DESC;
```

```
SELECT
    COUNT(*) FILTER (WHERE degree IS NULL) AS missing_degree,
    COUNT(*) FILTER (WHERE institution IS NULL) AS
missing_institution,
    COUNT(*) FILTER (WHERE major IS NULL) AS missing_major,
    COUNT(*) FILTER (WHERE country IS NULL) AS missing_country
FROM learners;
```

```
SELECT
    learner_id,
    COUNT(*) AS duplicate_count
FROM learners
GROUP BY learner_id
HAVING COUNT(*) > 1
ORDER BY duplicate_count DESC;
```

```
SELECT
    TRIM(LOWER(country)) AS country_clean,
    TRIM(LOWER(degree)) AS degree_clean,
    TRIM(LOWER(institution)) AS institution_clean,
    TRIM(LOWER(major)) AS major_clean,
    COUNT(*) AS duplicate_count
FROM learners
GROUP BY
    TRIM(LOWER(country)),
    TRIM(LOWER(degree)),
    TRIM(LOWER(institution)),
    TRIM(LOWER(major))
HAVING COUNT(*) > 1
ORDER BY duplicate_count DESC;
```

```
SELECT
```

```

-- Missing values
COUNT(*) FILTER (
    WHERE country IS NULL OR TRIM(LOWER(country)) IN ('null', '')
) AS missing_country,

COUNT(*) FILTER (
    WHERE degree IS NULL OR TRIM(LOWER(degree)) IN ('null', '')
) AS missing_degree,

COUNT(*) FILTER (
    WHERE institution IS NULL OR TRIM(LOWER(institution)) IN
('null', '')
) AS missing_institution,

COUNT(*) FILTER (
    WHERE major IS NULL OR TRIM(LOWER(major)) IN ('null', '')
) AS missing_major,

-- Duplicate combinations (country+degree+institution+major)
(
    SELECT COUNT(*)
    FROM (
        SELECT
            TRIM(LOWER(country)) AS country_clean,
            TRIM(LOWER(degree)) AS degree_clean,
            TRIM(LOWER(institution)) AS institution_clean,
            TRIM(LOWER(major)) AS major_clean,
            COUNT(*) AS dup_count
        FROM learners
        GROUP BY
            TRIM(LOWER(country)),
            TRIM(LOWER(degree)),
            TRIM(LOWER(institution)),
            TRIM(LOWER(major))
        HAVING COUNT(*) > 1
    ) AS dup_groups
) AS duplicate_groups
FROM learners;

```

```

SELECT
    -- Missing counts
    COUNT(*) FILTER (
        WHERE country IS NULL OR TRIM(LOWER(country)) IN ('null', '')
    ) AS missing_country,

    COUNT(*) FILTER (
        WHERE degree IS NULL OR TRIM(LOWER(degree)) IN ('null', '')
    ) AS missing_degree,

    COUNT(*) FILTER (

```

```

        WHERE institution IS NULL OR TRIM(LOWER(institution)) IN
('null', '')
    ) AS missing_institution,

    COUNT(*) FILTER (
        WHERE major IS NULL OR TRIM(LOWER(major)) IN ('null', '')
    ) AS missing_major,

    -- Total duplicate rows (across all columns)
    (
        SELECT COUNT(*)
        FROM learners
        WHERE (learner_id, country, degree, institution, major) IN (
            SELECT
                learner_id, country, degree, institution, major
            FROM learners
            GROUP BY
                learner_id, country, degree, institution, major
            HAVING COUNT(*) > 1
        )
    ) AS total_duplicate_rows
FROM learners;

```

```

SELECT country, COUNT(*) AS learners_count
FROM learners
GROUP BY country
ORDER BY learners_count DESC;

```

```

SELECT country, COUNT(*) AS total_learners
FROM learners
GROUP BY country
ORDER BY total_learners DESC;

```

```

SELECT country, degree, COUNT(*) AS total
FROM learners
GROUP BY country, degree
ORDER BY total DESC
limit 10;

```

```

SELECT
    degree,
    major,
    COUNT(*) AS total_students
FROM learners
GROUP BY degree, major
ORDER BY degree, total_students DESC
limit 10;

```

```

WITH top_countries AS (
    SELECT country
    FROM learners
    WHERE country IS NOT NULL AND TRIM(country) <> ''

```

```

        GROUP BY country
        ORDER BY COUNT(*) DESC
        LIMIT 5
    )
    -- Count learners per Major for only those Top Countries
    SELECT
        l.country,
        l.major,
        COUNT(*) AS total_students
    FROM learners l
    JOIN top_countries tc ON l.country = tc.country
    WHERE l.major IS NOT NULL AND TRIM(l.major) <> ''
    GROUP BY l.country, l.major
    ORDER BY l.country, total_students DESC;

```

2. COGNITO DATASET

```

SELECT * FROM public."Cognito_Raw2"
ORDER BY user_id ASC

```

```

SELECT COUNT(*) AS total_rows
FROM public."Cognito_Raw2"

```

```

SELECT COUNT(*) AS total_columns
FROM information_schema.columns
WHERE table_name = 'Cognito_Raw2'
    AND table_schema = 'public';

```

```

SELECT
    SUM(CASE WHEN user_id = 'NULL' THEN 1 ELSE 0 END) AS null_user_id,
    SUM(CASE WHEN email = 'NULL' THEN 1 ELSE 0 END) AS null_email,
    SUM(CASE WHEN gender = 'NULL' THEN 1 ELSE 0 END) AS null_gender,
    SUM(CASE WHEN usercreatedate = 'NULL' THEN 1 ELSE 0 END) AS
null_usercreatedate,
    SUM(CASE WHEN userlastmodifieddate = 'NULL' THEN 1 ELSE 0 END) AS
null_userlastmodifieddate,
    SUM(CASE WHEN birthdate = 'NULL' THEN 1 ELSE 0 END) AS
null_birthdate,
    SUM(CASE WHEN city = 'NULL' THEN 1 ELSE 0 END) AS null_city,
    SUM(CASE WHEN zip = 'NULL' THEN 1 ELSE 0 END) AS null_zip,
    SUM(CASE WHEN state = 'NULL' THEN 1 ELSE 0 END) AS null_state
FROM public."Cognito_Raw2"

```

```

SELECT
    gender,
    COUNT(*) AS count
FROM public."Cognito_Raw2"

```

```
WHERE gender IN ('Male', 'Female', 'Other', 'Don%27t want to
specify')
GROUP BY gender;
```

```
SELECT
    city,
    COUNT(*) AS user_count
FROM public."Cognito_Raw2"
WHERE city IS NOT NULL
    AND city != 'NULL'
GROUP BY city
ORDER BY user_count DESC
LIMIT 10;
```

```
SELECT
    state,
    COUNT(*) AS user_count
FROM public."Cognito_Raw2"
WHERE state IS NOT NULL
    AND state != 'NULL'
GROUP BY state
ORDER BY user_count DESC
LIMIT 10;
```

```
SELECT
    TO_DATE(LEFT(usercreateddate, 10), 'YYYY-MM-DD') AS creation_date,
    COUNT(*) AS user_count
FROM public."Cognito_Raw2"
WHERE usercreateddate IS NOT NULL
    AND usercreateddate != 'NULL'
    AND LEFT(usercreateddate, 10) ~ '^\\d{4}-\\d{2}-\\d{2}$'
GROUP BY creation_date
ORDER BY creation_date;
```

```
SELECT
    TO_DATE(LEFT(userlastmodifieddate, 10), 'YYYY-MM-DD') AS
last_modified_date,
    COUNT(*) AS user_count
FROM public."Cognito_Raw2"
WHERE userlastmodifieddate IS NOT NULL
    AND userlastmodifieddate != 'NULL'
    AND LEFT(userlastmodifieddate, 10) ~ '^\\d{4}-\\d{2}-\\d{2}$'
GROUP BY last_modified_date
ORDER BY last_modified_date;
```



```

SELECT
    COUNT(*) AS total_rows,
    COUNT(*) - COUNT(user_id) + COUNT(*) FILTER (WHERE user_id =
'NULL') AS missing_user_id,
    COUNT(*) - COUNT(email) + COUNT(*) FILTER (WHERE email = 'NULL') AS
missing_email,
    COUNT(*) - COUNT(gender) + COUNT(*) FILTER (WHERE gender = 'NULL')
AS missing_gender,
    COUNT(*) - COUNT(usercreatedate) + COUNT(*) FILTER (WHERE
usercreatedate = 'NULL') AS missing_usercreatedate,
    COUNT(*) - COUNT(userlastmodifieddate) + COUNT(*) FILTER (WHERE
userlastmodifieddate = 'NULL') AS missing_userlastmodifieddate,
    COUNT(*) - COUNT(birthdate) + COUNT(*) FILTER (WHERE birthdate =
'NULL') AS missing_birthdate,
    COUNT(*) - COUNT(city) + COUNT(*) FILTER (WHERE city = 'NULL') AS
missing_city,
    COUNT(*) - COUNT(zip) + COUNT(*) FILTER (WHERE zip = 'NULL') AS
missing_zip,
    COUNT(*) - COUNT(state) + COUNT(*) FILTER (WHERE state = 'NULL') AS
missing_state
FROM public."Cognito_Raw2";

```

```

SELECT
    *,
    COUNT(*) AS duplicate_count
FROM public."Cognito_Raw2"
GROUP BY
    user_id, email, gender, usercreatedate, userlastmodifieddate,
    birthdate, city, zip, state
HAVING COUNT(*) > 1
ORDER BY duplicate_count DESC;

```

```

SELECT
    MIN(city) AS first_city_alphabetically,
    MAX(city) AS last_city_alphabetically
FROM public."Cognito_Raw2"
WHERE city IS NOT NULL;

```

```

SELECT
    CASE
        WHEN EXTRACT(YEAR FROM AGE(CURRENT_DATE, TO_DATE(birthdate,
'MM/DD/YYYY')) < 13 THEN 'Child'
        WHEN EXTRACT(YEAR FROM AGE(CURRENT_DATE, TO_DATE(birthdate,
'MM/DD/YYYY')) BETWEEN 13 AND 19 THEN 'Teenager'
        WHEN EXTRACT(YEAR FROM AGE(CURRENT_DATE, TO_DATE(birthdate,
'MM/DD/YYYY')) BETWEEN 20 AND 59 THEN 'Adult'
        ELSE 'Old'
    END AS age_group_label,

    CASE

```

```

        WHEN EXTRACT(YEAR FROM AGE(CURRENT_DATE, TO_DATE(birthdate,
'MM/DD/YYYY')) < 13 THEN '<13'
        WHEN EXTRACT(YEAR FROM AGE(CURRENT_DATE, TO_DATE(birthdate,
'MM/DD/YYYY')) BETWEEN 13 AND 19 THEN '13-19'
        WHEN EXTRACT(YEAR FROM AGE(CURRENT_DATE, TO_DATE(birthdate,
'MM/DD/YYYY')) BETWEEN 20 AND 59 THEN '20-59'
        ELSE '60+'
    END AS age_range,

    COUNT(*) AS user_count
FROM public."Cognito_Raw2"
WHERE birthdate ~ '^\\d{1,2}/\\d{1,2}/\\d{4}$'
GROUP BY age_group_label, age_range
ORDER BY age_group_label;

```

```

SELECT
    CASE
        WHEN age < 13 THEN 'Child'
        WHEN age BETWEEN 13 AND 19 THEN 'Teenager'
        WHEN age BETWEEN 20 AND 59 THEN 'Adult'
        ELSE 'Old'
    END AS age_group_label,

    CASE
        WHEN age < 13 THEN '<13'
        WHEN age BETWEEN 13 AND 19 THEN '13-19'
        WHEN age BETWEEN 20 AND 59 THEN '20-59'
        ELSE '60+'
    END AS age_range,

    COUNT(*) AS user_count,
    ROUND(AVG(age), 2) AS mean_age,
    MODE() WITHIN GROUP (ORDER BY age) AS mode_age
FROM (
    SELECT
        EXTRACT(YEAR FROM AGE(CURRENT_DATE, TO_DATE(birthdate,
'MM/DD/YYYY'))::int AS age
        FROM public."Cognito_Raw2"
        WHERE birthdate ~ '^\\d{1,2}/\\d{1,2}/\\d{4}$'
    ) sub
GROUP BY age_group_label, age_range
ORDER BY age_group_label;

```

```

WITH age_data AS (
    SELECT

```

```

        EXTRACT(YEAR FROM AGE(CURRENT_DATE, TO_DATE(birthdate,
'MM/DD/YYYY')))) AS age
        FROM public."Cognito_Raw2"
        WHERE birthdate ~ '^\\d{1,2}/\\d{1,2}/\\d{4}$'
    )
SELECT
    ROUND(AVG(age), 2) AS mean_age,
    MODE() WITHIN GROUP (ORDER BY age) AS mode_age,
    ROUND(STDDEV(age), 2) AS standarddddeviation_age
FROM age_data;

```

3. MARKETING DATASET

```

-- ~~~~~(IMPORTING DATASET)~~~~~
-- #####Creating Marketing Table

CREATE TABLE marketing_campaign (
    ad_account_name VARCHAR(100),
    campaign_name VARCHAR(255),
    delivery_status VARCHAR(50),
    delivery_level VARCHAR(50),
    reach INTEGER,
    outbound_clicks INTEGER,
    outbound_type INTEGER,
    result_type VARCHAR(100),
    results INTEGER,
    cost_per_result NUMERIC(10,6),
    amount_spent_aed NUMERIC(10,2),
    cpc NUMERIC(10,6),
    reporting_starts DATE
);

COPY marketing_campaign(
    ad_account_name,
    campaign_name,
    delivery_status,
    delivery_level,
    reach,
    outbound_clicks,
    outbound_type,
    result_type,
    results,
    cost_per_result,
    amount_spent_aed,
    cpc,
    reporting_starts
)
FROM 'C:/Program Files/PostgreSQL/17/data/Marketing Campaign Data All
Accounts (2023-2024) (Detail1).csv'
DELIMITER ','

```

```

CSV HEADER
ENCODING 'LATIN1';

-- Running Query to check the Import
select * from marketing_campaign limit 5;

-- Datatype of Each column
SELECT column_name, data_type
FROM information_schema.columns
WHERE table_name = 'marketing_campaign';

-- ~~~~~(ROWS AND COLUMNS)~~~~~
-- ##### Checking Rows and Cols

-- number of rows
SELECT
    r.row_count,
    c.column_count
FROM
    (SELECT COUNT(*) AS row_count FROM marketing_campaign) r
CROSS JOIN
    (SELECT COUNT(*) AS column_count
     FROM information_schema.columns
     WHERE table_name = 'marketing_campaign') c;
-- checking last rows
SELECT COUNT(*) FROM marketing_campaign;
SELECT *
FROM marketing_campaign
ORDER BY ad_account_name DESC
LIMIT 10;
-- last 7 rows are null so deleting null rows
DELETE FROM marketing_campaign
WHERE ad_account_name IS NULL;
-- checking last 5 rows again
SELECT *
FROM marketing_campaign
ORDER BY ad_account_name DESC
LIMIT 5;
-- names of columns
SELECT column_name
FROM information_schema.columns
WHERE table_name = 'marketing_campaign';

-- ~~~~~(DATA CLEANING)~~~~~

-- #####checking null values in each column

SELECT string_agg(
    format('COUNT(*) - COUNT(%I) AS %I_nulls', column_name,
    column_name),
    ','
) AS null_count_query

```

```

FROM information_schema.columns
WHERE table_name = 'marketing_campaign';
-- copy pasting output of previous query
SELECT
COUNT(*) - COUNT(reporting_starts) AS reporting_starts_nulls,
COUNT(*) - COUNT(outbound_type) AS outbound_type_nulls,
COUNT(*) - COUNT(results) AS results_nulls,
COUNT(*) - COUNT(cost_per_result) AS cost_per_result_nulls,
COUNT(*) - COUNT(amount_spent_aed) AS amount_spent_aed_nulls,
COUNT(*) - COUNT(cpc) AS cpc_nulls,
COUNT(*) - COUNT(reach) AS reach_nulls,
COUNT(*) - COUNT(outbound_clicks) AS outbound_clicks_nulls,
COUNT(*) - COUNT(campaign_name) AS campaign_name_nulls,
COUNT(*) - COUNT(delivery_status) AS delivery_status_nulls,
COUNT(*) - COUNT(delivery_level) AS delivery_level_nulls,
COUNT(*) - COUNT(ad_account_name) AS ad_account_name_nulls,
COUNT(*) - COUNT(result_type) AS result_type_nulls
FROM marketing_campaign;
-- cleaning null values
-- 1. cleaning campaign_name column
SELECT *
FROM marketing_campaign
WHERE campaign_name IS NULL;

UPDATE marketing_campaign
SET campaign_name = 'Unnamed'
WHERE campaign_name IS NULL;

SELECT *
FROM marketing_campaign
WHERE campaign_name = 'Unnamed';

SELECT *
FROM marketing_campaign
WHERE campaign_name IS NULL;
-- 2. cleaning outbound_clicks column
SELECT *
FROM marketing_campaign
WHERE outbound_clicks IS NULL;
-- the column outbound_clicks, outbound_type and cpc have 2 blank
enteries and all of them belong to same rows
-- thats why filling the null values with 0
UPDATE marketing_campaign
SET
    outbound_type = COALESCE(outbound_type, 0),
    outbound_clicks = COALESCE(outbound_clicks, 0),
    cpc = COALESCE(cpc, 0)
WHERE outbound_type IS NULL
    OR outbound_clicks IS NULL
    OR cpc IS NULL;
-- checking update
SELECT *

```

```

FROM marketing_campaign
WHERE outbound_clicks = 0;
-- checking all null values once again.
SELECT
COUNT(*) - COUNT(reporting_starts) AS reporting_starts_nulls,
COUNT(*) - COUNT(outbound_type) AS outbound_type_nulls,
COUNT(*) - COUNT(results) AS results_nulls,
COUNT(*) - COUNT(cost_per_result) AS cost_per_result_nulls,
COUNT(*) - COUNT(amount_spent_aed) AS amount_spent_aed_nulls,
COUNT(*) - COUNT(cpc) AS cpc_nulls,
COUNT(*) - COUNT(reach) AS reach_nulls,
COUNT(*) - COUNT(outbound_clicks) AS outbound_clicks_nulls,
COUNT(*) - COUNT(campaign_name) AS campaign_name_nulls,
COUNT(*) - COUNT(delivery_status) AS delivery_status_nulls,
COUNT(*) - COUNT(delivery_level) AS delivery_level_nulls,
COUNT(*) - COUNT(ad_account_name) AS ad_account_name_nulls,
COUNT(*) - COUNT(result_type) AS result_type_nulls
FROM marketing_campaign;
-- the table is now free from null values

-- #####Checking for duplicates rows
SELECT COUNT(*) AS duplicate_rows
FROM (
    SELECT campaign_name
    FROM marketing_campaign
    GROUP BY campaign_name, reach, cpc
    HAVING COUNT(*) > 1
) sub;
--there is no duplicate rows in marketing dataset

-- fixing problematic row with reach 0
-- replacing the 0 value with median
SELECT *
FROM marketing_campaign
WHERE reach = 0;
UPDATE marketing_campaign
SET reach = sub.median_reach
FROM (
    SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY reach) AS
median_reach
    FROM marketing_campaign
    WHERE reach <> 0
) sub
WHERE reach = 0;

-- ~~~~~(Data Description)~~~~~

-- For numericals value
SELECT string_agg(format(
    $f$
    SELECT
        %L AS column_name,

```

```

        MIN(%I) AS min_value,
        MAX(%I) AS max_value,
        AVG(%I) AS avg_value,
        STDDEV(%I) AS stddev_value,
        PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY %I) AS
median_value,
        MODE() WITHIN GROUP (ORDER BY %I) AS mode_value
FROM marketing_campaign
    $$,
    column_name, column_name, column_name, column_name, column_name,
column_name, column_name
), ' UNION ALL '
FROM information_schema.columns
WHERE table_name = 'marketing_campaign'
    AND data_type IN ('integer', 'numeric', 'real', 'double
precision');
-- copy pasting output of previous query
SELECT
    'reach' AS column_name,
    MIN(reach) AS min_value,
    MAX(reach) AS max_value,
    AVG(reach) AS avg_value,
    STDDEV(reach) AS stddev_value,
    PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY reach) AS
median_value,
    MODE() WITHIN GROUP (ORDER BY reach) AS mode_value
FROM marketing_campaign
    UNION ALL
SELECT
    'outbound_clicks' AS column_name,
    MIN(outbound_clicks) AS min_value,
    MAX(outbound_clicks) AS max_value,
    AVG(outbound_clicks) AS avg_value,
    STDDEV(outbound_clicks) AS stddev_value,
    PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY outbound_clicks) AS
median_value,
    MODE() WITHIN GROUP (ORDER BY outbound_clicks) AS mode_value
FROM marketing_campaign
    UNION ALL
SELECT
    'outbound_type' AS column_name,
    MIN(outbound_type) AS min_value,
    MAX(outbound_type) AS max_value,
    AVG(outbound_type) AS avg_value,
    STDDEV(outbound_type) AS stddev_value,
    PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY outbound_type) AS
median_value,
    MODE() WITHIN GROUP (ORDER BY outbound_type) AS mode_value
FROM marketing_campaign
    UNION ALL
SELECT
    'results' AS column_name,

```

```

        MIN(results) AS min_value,
        MAX(results) AS max_value,
        AVG(results) AS avg_value,
        STDDEV(results) AS stddev_value,
        PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY results) AS
median_value,
        MODE() WITHIN GROUP (ORDER BY results) AS mode_value
FROM marketing_campaign
UNION ALL
SELECT
    'cost_per_result' AS column_name,
    MIN(cost_per_result) AS min_value,
    MAX(cost_per_result) AS max_value,
    AVG(cost_per_result) AS avg_value,
    STDDEV(cost_per_result) AS stddev_value,
    PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY cost_per_result) AS
median_value,
    MODE() WITHIN GROUP (ORDER BY cost_per_result) AS mode_value
FROM marketing_campaign
UNION ALL
SELECT
    'amount_spent_aed' AS column_name,
    MIN(amount_spent_aed) AS min_value,
    MAX(amount_spent_aed) AS max_value,
    AVG(amount_spent_aed) AS avg_value,
    STDDEV(amount_spent_aed) AS stddev_value,
    PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY amount_spent_aed)
AS median_value,
    MODE() WITHIN GROUP (ORDER BY amount_spent_aed) AS mode_value
FROM marketing_campaign
UNION ALL
SELECT
    'cpc' AS column_name,
    MIN(cpc) AS min_value,
    MAX(cpc) AS max_value,
    AVG(cpc) AS avg_value,
    STDDEV(cpc) AS stddev_value,
    PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY cpc) AS
median_value,
    MODE() WITHIN GROUP (ORDER BY cpc) AS mode_value
FROM marketing_campaign;

-- for catagorical data
-- getting catagorical columns name
SELECT column_name, data_type
FROM information_schema.columns
WHERE table_name = 'marketing_campaign'
AND data_type IN ('character varying', 'text', 'char');
-- for ad_account name
SELECT ad_account_name AS value, COUNT(*) AS value_count
FROM marketing_campaign
GROUP BY ad_account_name

```



```

ORDER BY value_count DESC;
-- for campaign_name
SELECT campaign_name AS value, COUNT(*) AS value_count
FROM marketing_campaign
GROUP BY campaign_name
ORDER BY value_count DESC;
-- for delivery_status
SELECT delivery_status AS value, COUNT(*) AS value_count
FROM marketing_campaign
GROUP BY delivery_status
ORDER BY value_count DESC;
-- for delivery_level
SELECT delivery_level AS value, COUNT(*) AS value_count
FROM marketing_campaign
GROUP BY delivery_level
ORDER BY value_count DESC;
-- for result_type
SELECT result_type AS value, COUNT(*) AS value_count
FROM marketing_campaign
GROUP BY result_type
ORDER BY value_count DESC;
-- for reporting_starts
SELECT reporting_starts AS value, COUNT(*) AS value_count
FROM marketing_campaign
GROUP BY reporting_starts
ORDER BY value_count DESC;

-- ~~~~~(DETECTING OUTLIERS)~~~~~
-- Getting numerical columns
SELECT column_name, data_type
FROM information_schema.columns
WHERE table_name = 'marketing_campaign'
      AND data_type IN ('smallint', 'integer', 'bigint',
                        'decimal', 'numeric', 'real', 'double
precision');
-- for ~~~~~(reach)~~~~~
-- no of outliers
WITH bounds AS (
    SELECT
        percentile_cont(0.25) WITHIN GROUP (ORDER BY reach) AS q1,
        percentile_cont(0.75) WITHIN GROUP (ORDER BY reach) AS q3,
        MIN(reach) AS min_val,
        MAX(reach) AS max_val
    FROM marketing_campaign
),
outliers AS (
    SELECT COUNT(*) AS outlier_count
    FROM marketing_campaign, bounds
    WHERE reach < bounds.q1 - 1.5 * (bounds.q3 - bounds.q1)
        OR reach > bounds.q3 + 1.5 * (bounds.q3 - bounds.q1)
)
SELECT

```

```

        b.min_val,
        b.max_val,
        b.q1 - 1.5 * (b.q3 - b.q1) AS lower_bound,
        b.q3 + 1.5 * (b.q3 - b.q1) AS upper_bound,
        o.outlier_count
FROM bounds b CROSS JOIN outliers o;
-- outliers rows
WITH bounds AS (
    SELECT
        percentile_cont(0.25) WITHIN GROUP (ORDER BY reach) AS q1,
        percentile_cont(0.75) WITHIN GROUP (ORDER BY reach) AS q3
    FROM marketing_campaign
)
SELECT *
FROM marketing_campaign, bounds
WHERE reach < bounds.q1 - 1.5 * (bounds.q3 - bounds.q1)
    OR reach > bounds.q3 + 1.5 * (bounds.q3 - bounds.q1)
ORDER BY reach DESC;
-- for ~~~~~(outbound_clicks)
WITH bounds AS (
    SELECT
        percentile_cont(0.25) WITHIN GROUP (ORDER BY outbound_clicks)
AS q1,
        percentile_cont(0.75) WITHIN GROUP (ORDER BY outbound_clicks)
AS q3,
        MIN(outbound_clicks) AS min_val,
        MAX(outbound_clicks) AS max_val
    FROM marketing_campaign
),
outliers AS (
    SELECT COUNT(*) AS outlier_count
    FROM marketing_campaign, bounds
    WHERE outbound_clicks < bounds.q1 - 1.5 * (bounds.q3 - bounds.q1)
        OR outbound_clicks > bounds.q3 + 1.5 * (bounds.q3 - bounds.q1)
)
SELECT
    b.min_val,
    b.max_val,
    b.q1 - 1.5 * (b.q3 - b.q1) AS lower_bound,
    b.q3 + 1.5 * (b.q3 - b.q1) AS upper_bound,
    o.outlier_count
FROM bounds b CROSS JOIN outliers o;
-- outliers rows
WITH bounds AS (
    SELECT
        percentile_cont(0.25) WITHIN GROUP (ORDER BY outbound_clicks)
AS q1,
        percentile_cont(0.75) WITHIN GROUP (ORDER BY outbound_clicks)
AS q3
    FROM marketing_campaign
)
SELECT *

```

```

FROM marketing_campaign, bounds
WHERE outbound_clicks < bounds.q1 - 1.5 * (bounds.q3 - bounds.q1)
    OR outbound_clicks > bounds.q3 + 1.5 * (bounds.q3 - bounds.q1);
-- for ~~~~~(outbound_type)~~~~~
WITH bounds AS (
    SELECT
        percentile_cont(0.25) WITHIN GROUP (ORDER BY outbound_type)
AS q1,
        percentile_cont(0.75) WITHIN GROUP (ORDER BY outbound_type)
AS q3,
        MIN(outbound_type) AS min_val,
        MAX(outbound_type) AS max_val
    FROM marketing_campaign
),
outliers AS (
    SELECT COUNT(*) AS outlier_count
    FROM marketing_campaign, bounds
    WHERE outbound_type < bounds.q1 - 1.5 * (bounds.q3 - bounds.q1)
        OR outbound_type > bounds.q3 + 1.5 * (bounds.q3 - bounds.q1)
)
SELECT
    b.min_val,
    b.max_val,
    b.q1 - 1.5 * (b.q3 - b.q1) AS lower_bound,
    b.q3 + 1.5 * (b.q3 - b.q1) AS upper_bound,
    o.outlier_count
FROM bounds b CROSS JOIN outliers o;
-- outliers rows
WITH bounds AS (
    SELECT
        percentile_cont(0.25) WITHIN GROUP (ORDER BY outbound_type)
AS q1,
        percentile_cont(0.75) WITHIN GROUP (ORDER BY outbound_type)
AS q3
    FROM marketing_campaign
)
SELECT *
FROM marketing_campaign, bounds
WHERE outbound_type < bounds.q1 - 1.5 * (bounds.q3 - bounds.q1)
    OR outbound_type > bounds.q3 + 1.5 * (bounds.q3 - bounds.q1);
-- for ~~~~~(results)~~~~~
WITH bounds AS (
    SELECT
        percentile_cont(0.25) WITHIN GROUP (ORDER BY results) AS q1,
        percentile_cont(0.75) WITHIN GROUP (ORDER BY results) AS q3,
        MIN(results) AS min_val,
        MAX(results) AS max_val
    FROM marketing_campaign),
outliers AS (
    SELECT COUNT(*) AS outlier_count
    FROM marketing_campaign, bounds
    WHERE results < bounds.q1 - 1.5 * (bounds.q3 - bounds.q1)

```

```

        OR results > bounds.q3 + 1.5 * (bounds.q3 - bounds.q1))
SELECT
    b.min_val,
    b.max_val,
    b.q1 - 1.5 * (b.q3 - b.q1) AS lower_bound,
    b.q3 + 1.5 * (b.q3 - b.q1) AS upper_bound,
    o.outlier_count
FROM bounds b CROSS JOIN outliers o;
-- outliers rows
WITH bounds AS (
    SELECT
        percentile_cont(0.25) WITHIN GROUP (ORDER BY results) AS q1,
        percentile_cont(0.75) WITHIN GROUP (ORDER BY results) AS q3
    FROM marketing_campaign
)
SELECT *
FROM marketing_campaign, bounds
WHERE results < bounds.q1 - 1.5 * (bounds.q3 - bounds.q1)
    OR results > bounds.q3 + 1.5 * (bounds.q3 - bounds.q1)
ORDER BY results DESC;
-- for ~~~~~~(cost_per_result)~~~~~
WITH bounds AS (
    SELECT
        percentile_cont(0.25) WITHIN GROUP (ORDER BY cost_per_result)
AS q1,
        percentile_cont(0.75) WITHIN GROUP (ORDER BY cost_per_result)
AS q3,
        MIN(cost_per_result) AS min_val,
        MAX(cost_per_result) AS max_val
    FROM marketing_campaign),
outliers AS (
    SELECT COUNT(*) AS outlier_count
    FROM marketing_campaign, bounds
    WHERE cost_per_result < bounds.q1 - 1.5 * (bounds.q3 - bounds.q1)
        OR cost_per_result > bounds.q3 + 1.5 * (bounds.q3 -
bounds.q1))
SELECT
    b.min_val,
    b.max_val,
    b.q1 - 1.5 * (b.q3 - b.q1) AS lower_bound,
    b.q3 + 1.5 * (b.q3 - b.q1) AS upper_bound,
    o.outlier_count
FROM bounds b CROSS JOIN outliers o;
-- outliers rows
WITH bounds AS (
    SELECT
        percentile_cont(0.25) WITHIN GROUP (ORDER BY cost_per_result)
AS q1,
        percentile_cont(0.75) WITHIN GROUP (ORDER BY cost_per_result)
AS q3
    FROM marketing_campaign
)

```

```

SELECT *
FROM marketing_campaign, bounds
WHERE cost_per_result < bounds.q1 - 1.5 * (bounds.q3 - bounds.q1)
      OR cost_per_result > bounds.q3 + 1.5 * (bounds.q3 - bounds.q1);
-- for ~~~~~~(amount_spent_aed)~~~~~
WITH bounds AS (
    SELECT
        percentile_cont(0.25) WITHIN GROUP (ORDER BY
amount_spent_aed) AS q1,
        percentile_cont(0.75) WITHIN GROUP (ORDER BY
amount_spent_aed) AS q3,
        MIN(amount_spent_aed) AS min_val,
        MAX(amount_spent_aed) AS max_val
    FROM marketing_campaign),
outliers AS (
    SELECT COUNT(*) AS outlier_count
    FROM marketing_campaign, bounds
    WHERE amount_spent_aed < bounds.q1 - 1.5 * (bounds.q3 -
bounds.q1)
      OR amount_spent_aed > bounds.q3 + 1.5 * (bounds.q3 -
bounds.q1))
SELECT
    b.min_val,
    b.max_val,
    b.q1 - 1.5 * (b.q3 - b.q1) AS lower_bound,
    b.q3 + 1.5 * (b.q3 - b.q1) AS upper_bound,
    o.outlier_count
FROM bounds b CROSS JOIN outliers o;
-- outliers rows
WITH bounds AS (
    SELECT
        percentile_cont(0.25) WITHIN GROUP (ORDER BY
amount_spent_aed) AS q1,
        percentile_cont(0.75) WITHIN GROUP (ORDER BY
amount_spent_aed) AS q3
    FROM marketing_campaign
)
SELECT *
FROM marketing_campaign, bounds
WHERE amount_spent_aed < bounds.q1 - 1.5 * (bounds.q3 - bounds.q1)
      OR amount_spent_aed > bounds.q3 + 1.5 * (bounds.q3 - bounds.q1);
-- for ~~~~~~(cpc)~~~~~
WITH bounds AS (
    SELECT
        percentile_cont(0.25) WITHIN GROUP (ORDER BY cpc) AS q1,
        percentile_cont(0.75) WITHIN GROUP (ORDER BY cpc) AS q3,
        MIN(cpc) AS min_val,
        MAX(cpc) AS max_val
    FROM marketing_campaign),
outliers AS (
    SELECT COUNT(*) AS outlier_count
    FROM marketing_campaign, bounds

```

```

        WHERE cpc < bounds.q1 - 1.5 * (bounds.q3 - bounds.q1)
           OR cpc > bounds.q3 + 1.5 * (bounds.q3 - bounds.q1))
SELECT
    b.min_val,
    b.max_val,
    b.q1 - 1.5 * (b.q3 - b.q1) AS lower_bound,
    b.q3 + 1.5 * (b.q3 - b.q1) AS upper_bound,
    o.outlier_count
FROM bounds b CROSS JOIN outliers o;
-- outliers rows
WITH bounds AS (
    SELECT
        percentile_cont(0.25) WITHIN GROUP (ORDER BY cpc) AS q1,
        percentile_cont(0.75) WITHIN GROUP (ORDER BY cpc) AS q3
    FROM marketing_campaign
)
SELECT *
FROM marketing_campaign, bounds
WHERE cpc < bounds.q1 - 1.5 * (bounds.q3 - bounds.q1)
    OR cpc > bounds.q3 + 1.5 * (bounds.q3 - bounds.q1);

-- ~~~~~(CALCULATING CORRELATION)~~~~~
SELECT
    corr(reach, outbound_clicks) AS corr_reach_outbound_clicks,
    corr(reach, results) AS corr_reach_results,
    corr(reach, cost_per_result) AS corr_reach_cost_per_result,
    corr(reach, amount_spent_aed) AS corr_reach_amount_spent_aed,
    corr(reach, cpc) AS corr_reach_cpc,
    corr(outbound_clicks, results) AS corr_outbound_clicks_results,
    corr(outbound_clicks, cost_per_result) AS
corr_outbound_clicks_cost_per_result,
    corr(outbound_clicks, amount_spent_aed) AS
corr_outbound_clicks_amount_spent_aed,
    corr(outbound_clicks, cpc) AS corr_outbound_clicks_cpc,
    corr(results, cost_per_result) AS corr_results_cost_per_result,
    corr(results, amount_spent_aed) AS corr_results_amount_spent_aed,
    corr(results, cpc) AS corr_results_cpc,
    corr(cost_per_result, amount_spent_aed) AS
corr_cost_per_result_amount_spent_aed,
    corr(cost_per_result, cpc) AS corr_cost_per_result_cpc,
    corr(amount_spent_aed, cpc) AS corr_amount_spent_aed_cpc
FROM marketing_campaign;

-- ~~~~~(KPIfor ad accounts)~~~~~
SELECT
    ad_account_name,
    AVG(cost_per_result) AS avg_cpc,
    SUM(amount_spent_aed) AS total_amount_spent,
    SUM(results)::float / NULLIF(SUM(reach), 0) AS conversion_rate
FROM marketing_campaign
GROUP BY ad_account_name
ORDER BY ad_account_name;

```

```

SELECT
    ad_account_name,
    AVG(reach) AS avg_reach,
    AVG(outbound_clicks) AS avg_outbound_clicks,
    AVG(results) AS avg_results
FROM marketing_campaign
GROUP BY ad_account_name;

```

```

SELECT
    ad_account_name,
    AVG(cost_per_result) AS avg_cost_per_result
FROM marketing_campaign
GROUP BY ad_account_name
ORDER BY ad_account_name;

```

4. LEARNER OPPORTUNITY DATASET

```

--(importing dataset)
CREATE TABLE enrollment (
    enrollment_id VARCHAR(100),
    learner_id VARCHAR(100),
    assigned_cohort VARCHAR(20),
    apply_date TIMESTAMP,
    status INTEGER
);

COPY enrollment (
    enrollment_id,
    learner_id,
    assigned_cohort,
    apply_date,
    status
)
FROM 'E:\LearnerOpportunity_Raw(in).csv'
DELIMITER ','
CSV HEADER NULL AS 'NULL'

-- Running query to check the import
SELECT *
FROM enrollment
LIMIT 10;

--CHECKING ROWS AND COLUMNS
--checking rows
SELECT COUNT(*) AS total_rows

```

```

FROM enrollment;
--checking columns
SELECT COUNT(*) AS total_columns
FROM INFORMATION_SCHEMA.COLUMNS
WHERE table_name = 'enrollment';

--CHECKING NULL VALUES IN EACH COLUMN
SELECT
    SUM(CASE WHEN enrollment_id IS NULL THEN 1 ELSE 0 END) AS
enrollment_id_missing,
    SUM(CASE WHEN learner_id IS NULL THEN 1 ELSE 0 END) AS
learner_id_missing,
    SUM(CASE WHEN assigned_cohort IS NULL THEN 1 ELSE 0 END) AS
assigned_cohort_missing,
    SUM(CASE WHEN apply_date IS NULL THEN 1 ELSE 0 END) AS
apply_date_missing,
    SUM(CASE WHEN status IS NULL THEN 1 ELSE 0 END) AS status_missing
FROM enrollment;

--CLEANING NULL VALUES
UPDATE enrollment
SET assigned_cohort = 'Unknown'
WHERE assigned_cohort IS NULL;

UPDATE enrollment
SET apply_date = CURRENT_DATE
WHERE apply_date IS NULL;

UPDATE enrollment
SET status = 0
WHERE status IS NULL;

--CHECKING FOR DUPLICATE ROWS
SELECT *, COUNT(*) AS occurrences
FROM enrollment
GROUP BY enrollment_id, learner_id, assigned_cohort, apply_date,
status -- include all columns here
HAVING COUNT(*) > 1;

```

5. COHORT DATASET

```

-- 1. Basic dataset overview
SELECT COUNT(*) AS total_records FROM CohortRaw;
SELECT column_name, data_type FROM information_schema.columns WHERE
table_name = 'CohortRaw';

-- 2. Summary statistics for numerical columns
SELECT
    MIN(size) AS min_size,

```



```

        MAX(size) AS max_size,
        AVG(size) AS avg_size,
        PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY size) AS median_size,
        STDDEV(size) AS stddev_size
FROM CohortRaw;

```

-- 3. Date range analysis

```

SELECT
    MIN(TO_TIMESTAMP(start_date/1000)) AS earliest_start_date,
    MAX(TO_TIMESTAMP(start_date/1000)) AS latest_start_date,
    MIN(TO_TIMESTAMP(end_date/1000)) AS earliest_end_date,
    MAX(TO_TIMESTAMP(end_date/1000)) AS latest_end_date
FROM CohortRaw;

```

-- 4. Missing values check

```

SELECT
    COUNT(*) - COUNT(cohort_id) AS missing_cohort_ids,
    COUNT(*) - COUNT(cohort_code) AS missing_codes,
    COUNT(*) - COUNT(start_date) AS missing_start_dates,
    COUNT(*) - COUNT(end_date) AS missing_end_dates,
    COUNT(*) - COUNT(size) AS missing_sizes
FROM CohortRaw;

```

-- 5. Duplicate check

```

SELECT cohort_code, COUNT(*)
FROM CohortRaw
GROUP BY cohort_code
HAVING COUNT(*) > 1;

```

-- 6. Cohort size distribution analysis

```

SELECT
    CASE
        WHEN size < 100 THEN 'Small (<100)'
        WHEN size BETWEEN 100 AND 1000 THEN 'Medium (100-1000)'
        WHEN size BETWEEN 1001 AND 10000 THEN 'Large (1001-10000)'
        ELSE 'Very Large (>10000)'
    END AS size_category,
    COUNT(*) AS count,
    ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM CohortRaw), 2) AS
percentage
FROM CohortRaw
GROUP BY size_category
ORDER BY MIN(size);

```

-- 7. Cohort duration analysis

```

SELECT
    CASE
        WHEN (end_date - start_date) = 0 THEN 'Single day'
        WHEN (end_date - start_date) <= 86400000 THEN '1 day'
        WHEN (end_date - start_date) <= 604800000 THEN '1 week'
        WHEN (end_date - start_date) <= 2592000000 THEN '1 month'
        WHEN (end_date - start_date) <= 7776000000 THEN '3 months'
        ELSE 'Longer than 3 months'
    END AS duration_category,

```

```

        COUNT(*) AS count
FROM CohortRaw
GROUP BY duration_category
ORDER BY MIN(end_date - start_date);

-- 8. Outlier detection using IQR
WITH size_stats AS (
    SELECT
        PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY size) AS q1,
        PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY size) AS q3,
        PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY size) -
        PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY size) AS iqr
    FROM CohortRaw
)
SELECT cohort_id, cohort_code, size
FROM CohortRaw, size_stats
WHERE size < q1 - 1.5*iqr OR size > q3 + 1.5*iqr
ORDER BY size DESC;

-- 9. Cohort code pattern analysis
SELECT
    CASE
        WHEN cohort_code ~ '^B[A-Z0-9]{6}$' THEN 'Standard 7-char code'
        WHEN cohort_code ~ '^B[A-Z0-9]{5}$' THEN 'Standard 6-char code'
        WHEN cohort_code ~ '^B[A-Z0-9]{7}$' THEN 'Standard 8-char code'
        ELSE 'Non-standard format'
    END AS code_pattern,
    COUNT(*) AS count
FROM CohortRaw
GROUP BY code_pattern;

```

6. OPPORTUNITY DATASET

```

SELECT * FROM public.opportunity_raw
ORDER BY opportunity_id ASC;

SELECT COUNT(*) AS total_rows
FROM public.opportunity_raw;

SELECT COUNT(*) AS total_columns
FROM information_schema.columns
WHERE table_name = 'opportunity_raw'
    AND table_schema = 'public';

SELECT
    COUNT(*) FILTER (WHERE opportunity_id IS NULL) AS
missing_opportunity_id,
    COUNT(*) FILTER (WHERE opportunity_name IS NULL) AS
missing_opportunity_name,

```

```

        COUNT(*) FILTER (WHERE category IS NULL) AS missing_category,
        COUNT(*) FILTER (WHERE opportunity_code IS NULL) AS
missing_opportunity_code,
        COUNT(*) FILTER (WHERE tracking_questions IS NULL) AS
missing_tracking_questions
FROM public.opportunity_raw;

```

```

SELECT
    (COUNT(opportunity_id) <> COUNT(DISTINCT opportunity_id)) AS
opportunity_id_has_duplicates,
    (COUNT(opportunity_name) <> COUNT(DISTINCT opportunity_name)) AS
opportunity_name_has_duplicates,
    (COUNT(category) <> COUNT(DISTINCT category)) AS
category_has_duplicates,
    (COUNT(opportunity_code) <> COUNT(DISTINCT opportunity_code)) AS
opportunity_code_has_duplicates,
    (COUNT(tracking_questions) <> COUNT(DISTINCT tracking_questions))
AS tracking_questions_has_duplicates
FROM public.opportunity_raw;

```

```

SELECT
    COUNT(CASE WHEN opportunity_id IS NULL THEN 1 END) AS
opportunity_id_null_count,
    COUNT(CASE WHEN opportunity_name IS NULL THEN 1 END) AS
opportunity_name_null_count,
    COUNT(CASE WHEN category IS NULL THEN 1 END) AS
category_null_count,
    COUNT(CASE WHEN opportunity_code IS NULL THEN 1 END) AS
opportunity_code_null_count,
    COUNT(CASE WHEN tracking_questions IS NULL THEN 1 END) AS
tracking_questions_null_count
FROM opportunity_raw;

```

-- EDA Part

```

SELECT category, COUNT(*) AS frequency
FROM opportunity_raw
GROUP BY category
ORDER BY frequency DESC;

```

```

SELECT opportunity_name, COUNT(*) AS repetition_count
FROM opportunity_raw
GROUP BY opportunity_name
HAVING COUNT(*) > 1
ORDER BY repetition_count DESC;

```

```

SELECT COUNT(*) AS count_null_text
FROM opportunity_raw
WHERE tracking_questions = 'NULL';

```

```
SELECT
    category,
    SUM(CASE WHEN tracking_questions IS NOT NULL AND tracking_questions
<> 'NULL' THEN 1 ELSE 0 END) AS with_valid_tracking_questions,
    SUM(CASE WHEN tracking_questions IS NULL OR tracking_questions =
'NULL' THEN 1 ELSE 0 END) AS without_valid_tracking_questions
FROM opportunity_raw
GROUP BY category
ORDER BY category;
```